Computação Distribuída com MPI

Prof Alcides/Prof Mario MPI

+ Python + Docker

Nome: Bruno Germanetti Ramalho RA:10426491

1. Arquitetura, funções e tipos de dados em MPI (Python)

1.1 Estrutura básica de um programa MPI

Uma aplicação MPI é composta por vários processos executando o mesmo programa, porém cada processo possui um identificador chamado rank. Esse identificador permite que cada processo execute comportamentos diferentes dentro do mesmo código.

Em Python, a forma mais comum de utilizar MPI é através da biblioteca mpi4py.

Estrutura básica de um programa MPI em Python:

```
from mpi4py import MPI
# 1. Inicializa o ambiente de execução MPI (feito ao importar mpi4py)
comm = MPI.COMM_WORLD
# Lógica do programa paralelo aqui...
size = comm.Get_size() rank
= comm.Get_rank()
print("Processo", rank, "de", size)
# 2. Finaliza o ambiente MPI (opcional; ao sair, mpi4py finaliza)
# MPI.Finalize()  # só se precisar encerrar explicitamente
```

Onde:
- size representa o número total de processos
- rank representa o identificador do processo atual

OBS: Em Python, a inicialização ocorre ao importar mpi4py. Caso precise de controle explícito use MPI.Init() / MPI.Finalize().

Antes de executar, precisamos primeiro da instalação da biblioteca mpi4py, a instalação pode ser feita em um ambiente virtual do Python (venv) ou instalar globalmente, como estamos usando container (Codespaces), faremos globalmente:

```
pip install mpi4py
```

Para executar na própria máquina, utilize:

```
mpirun -np 4 python programa.py ou
mpirun --oversubscribe -np 4 python programa.py
```

Para executar nos containers, utilize a partir do Codespaces (host):

#ativando os containers e o ssh

```
docker compose up -d
docker compose exec master service ssh start
docker compose exec worker1 service ssh start
docker compose exec worker2 service ssh start
docker compose exec worker3 service ssh start
```

#instalar o pip nos containers:

```
docker compose exec master apt update
docker compose exec master apt install -y python3-pip

docker compose exec worker1 apt update
docker compose exec worker1 apt install -y python3-pip

docker compose exec worker2 apt update
docker compose exec worker2 apt install -y python3-pip

docker compose exec worker3 apt update
docker compose exec worker3 apt install -y python3-pip
```

#instalar o mpi4py nos containers:

```
docker compose exec master python3 -m pip install mpi4py
docker compose exec worker1 python3 -m pip install mpi4py
docker compose exec worker2 python3 -m pip install mpi4py
docker compose exec worker3 python3 -m pip install mpi4py
```

#copiando o arquivo para os containers

```
docker cp programa.py master:/home/mpiuser/ docker
cp programa.py worker1:/home/mpiuser/ docker cp
programa.py worker2:/home/mpiuser/ docker cp
programa.py worker3:/home/mpiuser/
```

Entrar no master e mudar para o usuário mpiuser, garantindo que exista o arquivo host antes de executar, também é necessário ter o arquivo que iremos rodar dentro de cada host.

```
docker compose exec master bash su
- mpiuser
```

```
mpirun --hostfile hosts -np 4 python3 programa.py
```

1.2 Funções fundamentais e seus parâmetros (Python)

- MPI.COMM_WORLD
  - Propósito: comunicador global com todos os processos do programa (equivale a MPI_COMM_WORLD em C). ○ Tipo: mpi4py.MPI.Comm.
- comm.Get_size()
  - Propósito: retorna o número total de processos no comunicador. ○ Parâmetros: nenhum.
  - Retorno: int (ex.: size = comm.Get_size()).
- comm.Get_rank() ○ Propósito: retorna o rank (ID) do processo chamador dentro do comunicador. ○ Parâmetros: nenhum.
  - Retorno: int (ex.: rank = comm.Get_rank()).
- MPI.Init() / MPI.Finalize() *(opcionais)*
  - Propósito: inicializar/finalizar explicitamente. Normalmente não são necessários porque o mpi4py gerencia isso ao importar/encerrar o programa.
  - Parâmetros: nenhum.
  - Retorno: None.

1.3 Funções de comunicação ponto a ponto (versão Python)

comm.send(obj, dest, tag=0)

- Propósito: envia uma mensagem (qualquer objeto Python serializável) para um processo específico.
- Parâmetros:
  - obj (*qualquer objeto Python*): o conteúdo da mensagem.
  - dest (*int*): rank do destino.
  - tag (*int*, opcional): etiqueta para distinguir mensagens.
- Retorno: None.
- Notas: equivalente conceitual a MPI_Send (C). Sujeito à mesma correspondência por tag/source/dest.

comm.recv(source=MPI.ANY_SOURCE, tag=MPI.ANY_TAG, status=None)

- Propósito: recebe uma mensagem; é bloqueante.
- Parâmetros:
  - source (*int* | MPI.ANY_SOURCE): de quem receber.
  - tag (*int* | MPI.ANY_TAG): etiqueta esperada. ○ status (MPI.Status | None): objeto preenchido com metadados (source real, tag real, contagem).
- Retorno: o objeto Python recebido.
- Notas: equivalente a MPI_Recv (C). Use status = MPI.Status() para consultar metadados.

Envio síncrono: comm.ssend(obj, dest, tag=0)

- Propósito: só completa quando o recv correspondente iniciou (elimina dependência de *buffering*).
- Equivalência: MPI_Ssend no C.

Troca 2-vias segura: comm.sendrecv(...)

- Propósito: envia e recebe em uma única chamada, evitando *deadlock* em trocas bidirecionais.
- Retorno: objeto recebido.

1.4 Tipos de dados em MPI (Python)

Em C você escolhe MPI_Datatype (ex.: MPI_INT, MPI_DOUBLE). Em Python há dois modos:

1. Alto nível (objetos Python): send/recv, bcast, scatter, gather, reduce aceitam objetos Python arbitrários (são serializados). É prático, porém pode ser menos eficiente para grandes volumes numéricos.
2. Baixo nível (buffers/nativos): para alto desempenho, use *buffers* (ex.: NumPy arrays) e as variantes bufferizadas (comm.Send, comm.Recv, comm.Sendrecv etc.), indicando tipos MPI nativos: MPI.INT, MPI.LONG, MPI.FLOAT, MPI.DOUBLE, MPI.BYTE… Ex.: um numpy.ndarray(dtype='float64') casa com MPI.DOUBLE.

OBS: Serialização no mpi4py, ele empacota o objeto para bytes, envia pelo MPI e, no outro lado, desempacota. Variantes bufferizadas é quando você passa buffers de memória contínua (ex.: numpy.ndarray, array do módulo array, bytearray) e usa as chamadas baixo nível que trabalham direto nos bytes.

1.5 Tabela de referência

MPI datatype (C)	Python alto nível	Python bufferizado (NumPy)
MPI_CHAR	str/bytes (serializado)	np.ndarray(dtype='int8'/'uint8') + MPI.CHAR/MPI.BYTE
MPI_INT	int (serializado)	np.ndarray(dtype='int32') + MPI.INT
MPI_LONG	int (serializado)	np.ndarray(dtype='int64') + MPI.LONG
MPI_FLOAT	float (serializado)	np.ndarray(dtype='float32') + MPI.FLOAT
MPI_DOUBLE	float (serializado)	np.ndarray(dtype='float64') + MPI.DOUBLE
MPI_BYTE	bytes	np.ndarray(dtype='uint8') + MPI.BYTE

OBS: se for protótipo/aprendizagem, comece com objetos Python (mais simples). Se for desempenho, use NumPy + tipos MPI.

1.6 Tipos de dados derivados (Python)

- Em C você usa MPI_Type_create_struct, MPI_Type_commit, MPI_Type_free para mapear estruturas/colunas não contíguas.
- Em Python, o caminho típico é usar NumPy e slices/views (colunas, *strides*) e transferi-las via chamadas bufferizadas (comm.Send/Recv).
- mpi4py também expõe MPI.Datatype.Create_struct (e similares) se você realmente precisar definir tipos derivados em baixo nível, mas em prática NumPy cobre a maioria dos casos de maneira mais simples e performática.

2. Padrões de comunicação em MPI

Em aplicações distribuídas é comum que vários processos precisem trocar informações entre si. Para facilitar essa comunicação, o MPI oferece operações coletivas, chamadas de padrões de comunicação, tópico já apresentado na última aula.

Os exercícios desta aula utilizam três padrões principais:

- Broadcast
- Gather
- Reduce

Esses padrões aparecem em diversas plataformas modernas de computação distribuída, como Hadoop, Spark e entre outras.

2.1 Broadcast (bcast)

O broadcast é utilizado quando um processo precisa enviar dados para todos os outros processos. Normalmente o processo responsável por enviar os dados é o rank 0, chamado também de processo mestre.

```
dados = comm.bcast(dados, root=0)
```

onde:

- dados: informação que será enviada e posteriormente retornar o dado enviado

- root: processo que possui os dados originais

Após a execução do broadcast todos os processos possuem acesso aos mesmos dados.

Esse padrão é muito utilizado para enviar:

- parâmetros de execução
- conjuntos de dados iniciais
- matrizes ou vetores de entrada

2.2 Gather (gather)

O gather é utilizado quando vários processos produzem resultados parciais que precisam ser reunidos em um processo final.

```
resultados = comm.gather(valor_local, root=0)
```

Cada processo envia seu valor local para o processo root.

Após a execução:

- o processo root recebe todos os resultados
- os outros processos recebem None

Esse padrão é muito utilizado quando cada processo executa parte de um cálculo e precisamos reunir os resultados finais.

2.3 Reduce (reduce)

O reduce é utilizado quando vários processos produzem valores que devem ser combinados através de uma operação matemática.

```
total = comm.reduce(valor_local, op=MPI.SUM, root=0)
```

Operações disponíveis:

operação	MPI
soma	MPI.SUM

máximo	MPI.MAX
mínimo	MPI.MIN
produto	MPI.PROD

O resultado final é entregue ao processo root.

Esse padrão é extremamente comum em algoritmos distribuídos.

3.
Exemplo “Olá Mundo” distribuído

Um exemplo simples de aplicação MPI é um programa que envia mensagens entre processos.

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank() if
rank != 0:
    mensagem = f"Saudações do processo {rank} de {size}"
comm.send(mensagem, dest=0) else:
    print(f"Saudações do processo {rank} de {size}")
for i in range(1, size):
        mensagem = comm.recv(source=i)
print(mensagem)
```

Neste exemplo:

- cada processo envia uma mensagem
- o processo rank 0 recebe todas as mensagens

Esse padrão é comum em aplicações distribuídas onde existe um processo coordenador.

3.1 Execução de programas MPI em Python
Para executar um programa MPI, todas as máquinas devem possuir o código e a infra para rodar MPI, depois executamos:

```
docker cp olamundo.py master:/home/mpiuser/ docker
cp olamundo.py worker1:/home/mpiuser/ docker cp
olamundo.py worker2:/home/mpiuser/ docker cp
olamundo.py worker3:/home/mpiuser/

docker compose exec master bash su
- mpiuser
mpirun --hostfile hosts -np 4 python3 olamundo.py

ou, para rodar no host base (Codespaces)sem entrar no
master
docker compose exec master su - mpiuser -c "mpirun -
hostfile hosts -np 4 python3 olamundo.py"
```

Cada processo executará uma instância do programa.

4.

Multiplicação de matrizes (versão sequencial)

A multiplicação de matrizes é um dos problemas clássicos usados para demonstrar paralelismo, pois cada linha da matriz resultado pode ser calculada independentemente. Essa característica torna o problema ideal para execução em sistemas distribuídos.

Fórmula: C[i][j] = Σ A[i][k] * B[k][j]

Implementação sequencial (rodar normalmente no python local)

```
import random
import time
N = 300
A = [[random.random() for _ in range(N)] for _ in range(N)]
B = [[random.random() for _ in range(N)] for _ in range(N)]
C = [[0]*N for _ in range(N)] inicio = time.time() for i in range(N):
  for j in range(N):         for k in range(N):
            C[i][j] += A[i][k] * B[k][j] fim
= time.time()
print("Tempo sequencial:", (fim-inicio)*1000, "ms")
```

Este algoritmo possui complexidade aproximada: $O(N^3)$

Execute normalmente:

```
python multiplicacaoMatriz.py
```

Multiplicação de matrizes (versão com Threads)

Uma forma de melhorar o desempenho é dividir o trabalho entre múltiplas threads, permitindo utilizar mais de um núcleo do processador. Cada thread será responsável por calcular um conjunto de linhas da matriz resultado.

```
import threading
import random
import time
N = 300
THREADS = 4
A = [[random.random() for _ in range(N)] for _ in range(N)]
B = [[random.random() for _ in range(N)] for _ in range(N)]
C = [[0]*N for _ in range(N)]
 def calcular(inicio,
fim):
    for i in range(inicio, fim):
        for j in range(N):
for k in range(N):
                C[i][j] += A[i][k] * B[k][j]
 inicio = time.time()
threads = [] linhas = N
```

5.

```
// THREADS for t in
range(THREADS):     ini
= t * linhas
    fim = N if t == THREADS-1 else (t+1)*linhas     th =
threading.Thread(target=calcular, args=(ini, fim))
threads.append(th)     th.start()
 for th in
threads:
    th.join()

fim = time.time()
print("Tempo com threads:", (fim-inicio)*1000, "ms")
```

Neste modelo utilizamos paralelismo dentro de uma única máquina.

Para executar, utilize:

```
python multiplicacaoMatrizThread.py
```

Como ficaria o código utilizando o MPI para distribuir o cálculo? E a questão do tempo, teríamos algum ganho?

Exercício 1 – Multiplicação de matrizes distribuída

No exemplo de multiplicação de matrizes deste material, vimos como dividir o cálculo utilizando threads. Em ambientes de alto desempenho é comum utilizar várias máquinas trabalhando juntas, formando um cluster computacional. Neste exercício você deverá implementar uma versão distribuída da multiplicação de matrizes utilizando MPI.

Objetivo

Distribuir o cálculo das linhas da matriz entre vários processos MPI.

Cada processo será responsável por calcular apenas parte da matriz final.

Regras

1. O processo rank 0 deve gerar as matrizes A e B.
2. As matrizes devem ser enviadas para todos os processos utilizando broadcast.
3. Cada processo calcula apenas algumas linhas da matriz resultado.
4. As partes calculadas devem ser enviadas para o processo 0 utilizando gather.
5. O processo 0 deve montar a matriz final.
6. O programa deve exibir o tempo de execução em milissegundos.

Dicas: Utilize as funções:

```
comm.Get_rank()
comm.Get_size()
comm.bcast()
comm.gather()
```

Execuções (testes)

Faça 3 testes, cada um com uma quantidade de elementos descrita abaixo, em seguida registre os tempos e envie também como parte da atividade (arq txt). Rode o código de matriz local (sem thread), depois o código com thread e posteriormente o código distribuído nos containers.

N = 300
N = 600
N= 1000

```
@solocr0w →/workspaces/Codewars (main) $ # 1. Sequencial (Local)
python3 matriz_seq.py 300

# 2. Threads (Local Paralelo)
python3 matriz_thr.py 300

# 3. Distribuído (Cluster)
docker compose exec master su - mpiuser -c "cd /home/mpiuser && mpirun --hostfile hosts -np 8 python3 matriz_dist.py 300"
Sequencial N=300: 5113.73 ms
Threads N=300: 2371.00 ms
Distribuído N=300: 2623.76 ms
```

```
@solocr0w →/workspaces/Codewars (main) $ # 1. Sequencial
python3 matriz_seq.py 600

# 2. Threads
python3 matriz_thr.py 600

# 3. Distribuído
docker compose exec master su - mpiuser -c "cd /home/mpiuser && mpirun --hostfile hosts -np 8 python3 matriz_dist.py 600"
Sequencial N=600: 46305.49 ms
Threads N=600: 21217.26 ms
Distribuído N=600: 21165.83 ms
```

```
@solocr0w →/workspaces/Codewars (main) $ # 1. Sequencial
python3 matriz_seq.py 600

# 2. Threads
python3 matriz_thr.py 600 …
Distribuído N=600: 21165.83 ms
@solocr0w →/workspaces/Codewars (main) $ # 1. Sequencial
python3 matriz_seq.py 1000

# 2. Threads
python3 matriz_thr.py 1000

# 3. Distribuído
docker compose exec master su - mpiuser -c "cd /home/mpiuser && mpirun --hostfile hosts -np 8 python3 matriz_dist.py 1000"
Sequencial N=1000: 211016.95 ms
Threads N=1000: 113516.64 ms
Distribuído N=1000: 94870.64 ms
```

Exercício 2 – Estimativa de π utilizando Monte Carlo

Ométodo de Monte Carlo é uma técnica que utiliza números aleatórios para estimar valores matemáticos. Neste exercício utilizaremos esse método para estimar o valor de π.

Considere um quadrado de lado 1 contendo um quarto de círculo.

Gerando pontos aleatórios dentro do quadrado:

- alguns pontos cairão dentro do círculo
- outros cairão fora

A proporção de pontos dentro do círculo aproxima: π / 4

Logo: π ≈ 4 * (pontos_dentro / pontos_totais)

Passos:

- Geração de pontos: Cada processo gera N pontos aleatórios (x, y) onde 0 <= x, y <= 1.

- Contagem local: Para cada ponto, verifique se x*x + y*y <= 1. Se for, o ponto está dentro do círculo. Conte quantos pontos locais estão dentro do círculo.
- Agregação de resultados: Use MPI_Send/MPI_Recv ou MPI_Reduce (indicado) para somar as contagens locais de todos os processos no processo 0.
- Cálculo final: No processo 0, calcule PI usando a fórmula PI = 4 * (total_pontos_dentro / total_pontos_gerados).
- Finalização MPI: Finalize o ambiente MPI.

Versão do código sequencial para teste com python local sem threads e sem MPI:

```
import random import
time N = 10000000
dentro = 0 inicio =
time.time() for _ in
range(N):
    x = random.random()      y =
random.random()      if x*x + y*y <= 1:
dentro += 1 pi = 4 * dentro / N fim =
time.time() print("PI aproximado:", pi)
print("Tempo:", (fim-inicio)*1000, "ms")
```

Para executar os exercícios:

```
docker cp exercicio<1 ou 2>.py master:/home/mpiuser/ docker
cp exercicio<1 ou 2>.py worker1:/home/mpiuser/ docker cp
exercicio<1 ou 2>.py worker2:/home/mpiuser/ docker cp
exercicio<1 ou 2>.py worker3:/home/mpiuser/

docker compose exec master bash su
- mpiuser
mpirun --hostfile hosts -np 4 python3 exercicio<1 ou 2>.py

ou, para rodar no host base (Codespaces) sem entrar no
master
docker compose exec master su - mpiuser -c "mpirun -
hostfile hosts -np 4 python3 exercicio<1 ou 2>.py"
```